

Semana 7 - Aspectos Práticos

João Florindo

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas - Brasil
florindo@unicamp.br

Outline

- 1 Ajuste de Hiperparâmetros
- 2 Batch Normalization
- 3 Classificação Multi-classes
- 4 Frameworks de Deep Learning

- Grande número de hiperparâmetros: α , β , β_1 , β_2 , ϵ , número de camadas e de unidades escondidas, decaimento da taxa de aprendizado, tamanho do *mini-batch*, etc.
- Alguns funcionam bem com valores padrão como $\beta = 0.9$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
- Prioridade: 1) α ; 2) de unidades escondidas, tamanho do *mini-batch* e β ; 3) número de camadas, decaimento da taxa de aprendizado, β_1 , β_2 , ϵ

- Grande número de hiperparâmetros: α , β , β_1 , β_2 , ϵ , número de camadas e de unidades escondidas, decaimento da taxa de aprendizado, tamanho do *mini-batch*, etc.
- Alguns funcionam bem com valores padrão como $\beta = 0.9$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
- Prioridade: 1) α ; 2) de unidades escondidas, tamanho do *mini-batch* e β ; 3) número de camadas, decaimento da taxa de aprendizado, β_1 , β_2 , ϵ

- Grande número de hiperparâmetros: α , β , β_1 , β_2 , ϵ , número de camadas e de unidades escondidas, decaimento da taxa de aprendizado, tamanho do *mini-batch*, etc.
- Alguns funcionam bem com valores padrão como $\beta = 0.9$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
- Prioridade: 1) α ; 2) de unidades escondidas, tamanho do *mini-batch* e β ; 3) número de camadas, decaimento da taxa de aprendizado, β_1 , β_2 , ϵ

Como escolher os valores?

- Usar um conjunto de referência: *dev set*
- No *machine learning* clássico era comum que se usasse *grid* de valores: OK com poucos hiperparâmetros
- No *deep learning* é preferível amostrar aleatoriamente: hiperparâmetros com importância diferente e desconhecida *a priori*

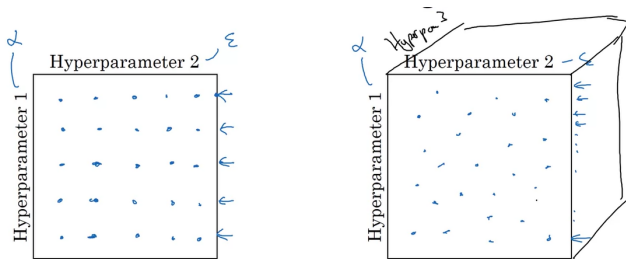
Como escolher os valores?

- Usar um conjunto de referência: *dev set*
- No *machine learning* clássico era comum que se usasse *grid* de valores: OK com poucos hiperparâmetros
- No *deep learning* é preferível amostrar aleatoriamente: hiperparâmetros com importância diferente e desconhecida *a priori*

Como escolher os valores?

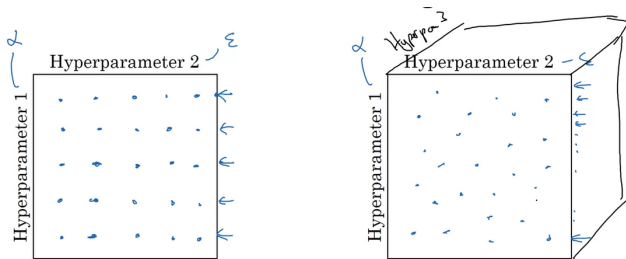
- Usar um conjunto de referência: *dev set*
- No *machine learning* clássico era comum que se usasse *grid* de valores: OK com poucos hiperparâmetros
- No *deep learning* é preferível amostrar aleatoriamente: hiperparâmetros com importância diferente e desconhecida *a priori*

Como escolher os valores?



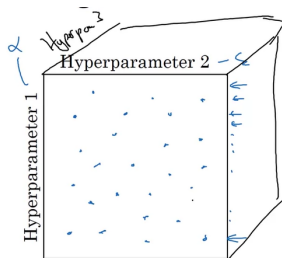
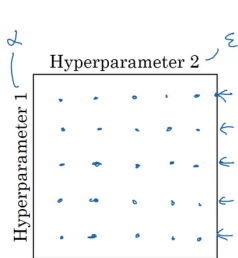
- Imagine que o Hiperparâmetro 1 seja α e o 2 seja ϵ (muito menos importante)
- No *grid*, testamos 5 valores de α e 5 de ϵ (que poderiam dar o mesmo resultado); i.e., só avaliamos 5 valores úteis
- Na busca aleatória, testamos efetivamente 25 valores diferentes de α

Como escolher os valores?



- Imagine que o Hiperparâmetro 1 seja α e o 2 seja ϵ (muito menos importante)
- No *grid*, testamos 5 valores de α e 5 de ϵ (que poderiam dar o mesmo resultado); i.e., só avaliamos 5 valores úteis
- Na busca aleatória, testamos efetivamente 25 valores diferentes de α

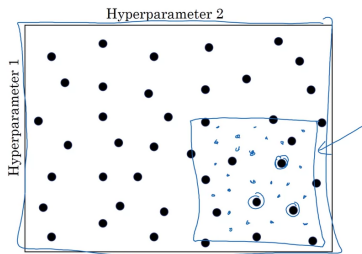
Como escolher os valores?



- Imagine que o Hiperparâmetro 1 seja α e o 2 seja ϵ (muito menos importante)
- No *grid*, testamos 5 valores de α e 5 de ϵ (que poderiam dar o mesmo resultado); i.e., só avaliamos 5 valores úteis
- Na busca aleatória, testamos efetivamente 25 valores diferentes de α

Como escolher os valores?

- Outra prática comum é ir refinando o espaço de busca:



- Quando se identifica uma região do conjunto valores com melhores resultados, dá-se um “*zoom in*” naquela região

Escala dos Valores

- Não necessariamente uma amostragem uniforme
- Na definição do número de neurônios entre 50 e 100 ou o número de camadas entre 2 e 4, uma amostragem uniforme é OK
- Porém, se queremos definir α entre 0.0001 e 1 uniformemente, temos apenas $\sim 10\%$ dos valores entre 0.0001 e 0.1
- SOLUÇÃO: **escala logarítmica!**

Escala dos Valores

- Não necessariamente uma amostragem uniforme
- Na definição do número de neurônios entre 50 e 100 ou o número de camadas entre 2 e 4, uma amostragem uniforme é OK
- Porém, se queremos definir α entre 0.0001 e 1 uniformemente, temos apenas $\sim 10\%$ dos valores entre 0.0001 e 0.1
- SOLUÇÃO: **escala logarítmica!**

Escala dos Valores

- Não necessariamente uma amostragem uniforme
- Na definição do número de neurônios entre 50 e 100 ou o número de camadas entre 2 e 4, uma amostragem uniforme é OK
- Porém, se queremos definir α entre 0.0001 e 1 uniformemente, temos apenas $\sim 10\%$ dos valores entre 0.0001 e 0.1
- SOLUÇÃO: escala logarítmica!

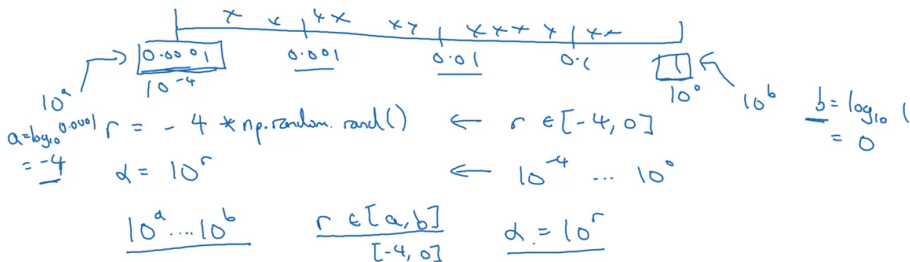
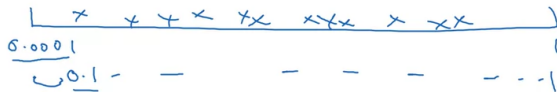
Escala dos Valores

- Não necessariamente uma amostragem uniforme
- Na definição do número de neurônios entre 50 e 100 ou o número de camadas entre 2 e 4, uma amostragem uniforme é OK
- Porém, se queremos definir α entre 0.0001 e 1 uniformemente, temos apenas $\sim 10\%$ dos valores entre 0.0001 e 0.1
- SOLUÇÃO: **escala logarítmica!**

Escala dos Valores

```
r = -4 * np.random.rand() # r em [-4, 0]
alpha = 10**r             # alpha em [10^-4, 1]
```

$$\alpha = 0.0001, \dots, 1$$



Truque para o β

- Se β está entre 0.9 e 0.999, amostramos $1 - \beta$ (entre 0.1 e 0.001)

$$\beta = 0.9 \quad \dots \quad 0.999$$

$\downarrow$
10

$\downarrow$
1000

$$1 - \beta = 0.1 \quad \dots \quad 0.001$$

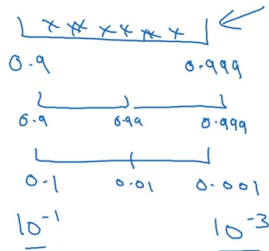
$$\beta: 0.9000 \rightarrow 0.9005 \} \sim 10$$

$$\beta: 0.999 \rightarrow 0.9995$$

~ 1000

~ 2000

$$\frac{1}{1 - \beta}$$



$$r \in [-3, -1]$$

$$1 - \beta = 10^r$$

$$\beta = 1 - 10^r$$

Truque para o β

- Quantidades parecidas de valores em $[0.9, 0.99]$ e $[0.99, 0.999]$
- Otimização muito sensível a mudanças mínimas para β próximo de 1
- EX.: Valores entre 0.9 e 0.9005 $\rightarrow \sim 10$ últimos valores
- 0.999 $\rightarrow \sim 1000$ dias; 0.9995 $\rightarrow \sim 2000$ dias (muito diferente!)

Truque para o β

- Quantidades parecidas de valores em $[0.9, 0.99]$ e $[0.99, 0.999]$
- Otimização muito sensível a mudanças mínimas para β próximo de 1
- EX.: Valores entre 0.9 e 0.9005 $\rightarrow \sim 10$ últimos valores
- 0.999 $\rightarrow \sim 1000$ dias; 0.9995 $\rightarrow \sim 2000$ dias (muito diferente!)

Truque para o β

- Quantidades parecidas de valores em $[0.9, 0.99]$ e $[0.99, 0.999]$
- Otimização muito sensível a mudanças mínimas para β próximo de 1
- EX.: Valores entre 0.9 e 0.9005 $\rightarrow \sim 10$ últimos valores
- 0.999 $\rightarrow \sim 1000$ dias; 0.9995 $\rightarrow \sim 2000$ dias (muito diferente!)

Truque para o β

- Quantidades parecidas de valores em $[0.9, 0.99]$ e $[0.99, 0.999]$
- Otimização muito sensível a mudanças mínimas para β próximo de 1
- EX.: Valores entre 0.9 e 0.9005 $\rightarrow \sim 10$ últimos valores
- 0.999 $\rightarrow \sim 1000$ dias; 0.9995 $\rightarrow \sim 2000$ dias (muito diferente!)

Observação Geral

- Mesmo amostragem uniforme poderá dar bons resultados, especialmente se usar refinamento

Reúso

- É comum a troca de métodos entre áreas do *deep learning*: EX.: CV e NLP
- Porém, configurações de hiperparâmetros geralmente não são transferidas
- Configuração pode se tornar obsoleta e é importante retestar de tempos em tempos

Reúso

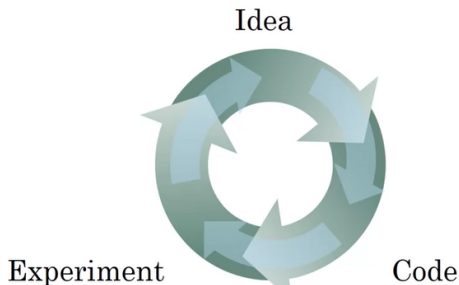
- É comum a troca de métodos entre áreas do *deep learning*: EX.: CV e NLP
- Porém, configurações de hiperparâmetros geralmente não são transferidas
- Configuração pode se tornar obsoleta e é importante retestar de tempos em tempos

Reúso

- É comum a troca de métodos entre áreas do *deep learning*: EX.: CV e NLP
- Porém, configurações de hiperparâmetros geralmente não são transferidas
- Configuração pode se tornar obsoleta e é importante retestar de tempos em tempos

Reúso

- É comum a troca de métodos entre áreas do *deep learning*: EX.: CV e NLP
- Porém, configurações de hiperparâmetros geralmente não são transferidas
- Configuração pode se tornar obsoleta e é importante retestar de tempos em tempos



Panda vs. Caviar

- Duas políticas de ajuste de hiperparâmetros: *babysitting* e paralelo
- *Babysitting* de um único modelo (ou um número bem pequeno)
- Observa a curva de aprendizado (e.g., a cada dia) e faz ajustes conforme o seu comportamento
- Mais usado quando se tem muito dado e pouco recurso para treinar

Panda vs. Caviar

- Duas políticas de ajuste de hiperparâmetros: *babysitting* e paralelo
- *Babysitting* de um único modelo (ou um número bem pequeno)
- Observa a curva de aprendizado (e.g., a cada dia) e faz ajustes conforme o seu comportamento
- Mais usado quando se tem muito dado e pouco recurso para treinar

Panda vs. Caviar

- Duas políticas de ajuste de hiperparâmetros: *babysitting* e paralelo
- *Babysitting* de um único modelo (ou um número bem pequeno)
- Observa a curva de aprendizado (e.g., a cada dia) e faz ajustes conforme o seu comportamento
- Mais usado quando se tem muito dado e pouco recurso para treinar

Panda vs. Caviar

- Duas políticas de ajuste de hiperparâmetros: *babysitting* e paralelo
- *Babysitting* de um único modelo (ou um número bem pequeno)
- Observa a curva de aprendizado (e.g., a cada dia) e faz ajustes conforme o seu comportamento
- Mais usado quando se tem muito dado e pouco recurso para treinar

Panda vs. Caviar

- Treinar vários modelos em paralelo, com diferentes configurações de hiperparâmetros
- Só é viável quando se tem um problema reduzido ou muito recurso computacional
- Analogia do *babysitting* com o panda: poucos filhos, investem tudo para que o filhote cresça bem
- Paralelo é o caviar: peixes depositam até 100 milhões de ovos e não ficam cuidando de cada um, apenas esperam que um (ou vários) cresça

Panda vs. Caviar

- Treinar vários modelos em paralelo, com diferentes configurações de hiperparâmetros
- Só é viável quando se tem um problema reduzido ou muito recurso computacional
- Analogia do *babysitting* com o panda: poucos filhos, investem tudo para que o filhote cresça bem
- Paralelo é o caviar: peixes depositam até 100 milhões de ovos e não ficam cuidando de cada um, apenas esperam que um (ou vários) cresça

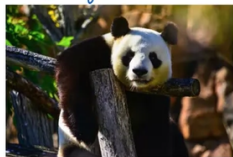
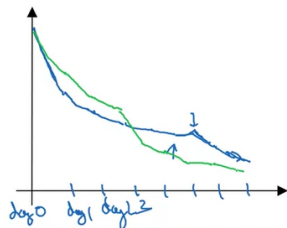
Panda vs. Caviar

- Treinar vários modelos em paralelo, com diferentes configurações de hiperparâmetros
- Só é viável quando se tem um problema reduzido ou muito recurso computacional
- Analogia do *babysitting* com o panda: poucos filhos, investem tudo para que o filhote cresça bem
- Paralelo é o caviar: peixes depositam até 100 milhões de ovos e não ficam cuidando de cada um, apenas esperam que um (ou vários) cresça

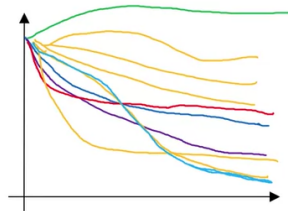
Panda vs. Caviar

- Treinar vários modelos em paralelo, com diferentes configurações de hiperparâmetros
- Só é viável quando se tem um problema reduzido ou muito recurso computacional
- Analogia do *babysitting* com o panda: poucos filhos, investem tudo para que o filhote cresça bem
- Paralelo é o caviar: peixes depositam até 100 milhões de ovos e não ficam cuidando de cada um, apenas esperam que um (ou vários) cresça

Panda vs. Caviar



Panda ←



Caviar ←

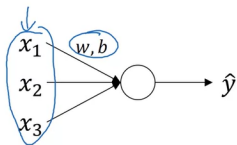
Outline

- 1 Ajuste de Hiperparâmetros
- 2 Batch Normalization**
- 3 Classificação Multi-classes
- 4 Frameworks de Deep Learning

- Algoritmo muito importante no crescimento do *deep learning* (Sergey Ioffe e Christian Szegedy)
- Torna a rede neural mais robusta, permite que um intervalo maior de hiperparâmetros funcione, facilitando o treinamento da rede
- Já vimos que, em um modelo como a regressão logística, normalizar as entradas acelera o treinamento

- Algoritmo muito importante no crescimento do *deep learning* (Sergey Ioffe e Christian Szegedy)
- Torna a rede neural mais robusta, permite que um intervalo maior de hiperparâmetros funcione, facilitando o treinamento da rede
- Já vimos que, em um modelo como a regressão logística, normalizar as entradas acelera o treinamento

- Algoritmo muito importante no crescimento do *deep learning* (Sergey Ioffe e Christian Szegedy)
- Torna a rede neural mais robusta, permite que um intervalo maior de hiperparâmetros funcione, facilitando o treinamento da rede
- Já vimos que, em um modelo como a regressão logística, normalizar as entradas acelera o treinamento



$$\mu = \frac{1}{n} \sum_i x^{(i)}$$

$$X = X - \mu$$

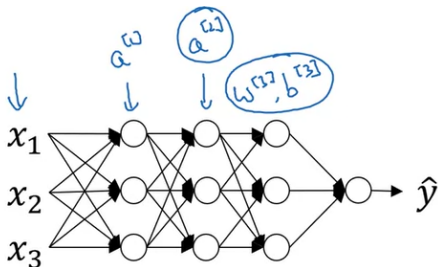
$$\sigma^2 = \frac{1}{n} \sum_i x^{(i)2}$$

$$X = X / \sigma$$

← elem. - wise



- Na rede profunda, normalizar $a^{[2]}$ acelera o aprendizado de $W^{[3]}, b^{[3]}$?



- É isso que faz o **batch normalization** (na verdade normaliza $z^{[2]}$)

- Porém, agora não necessariamente queremos média 0 e variância 1
- EX.: Na sigmoide $\sigma(z)$ não é interessante que z tenha variância 1

- Porém, agora não necessariamente queremos média 0 e variância 1
- EX.: Na sigmoide $\sigma(z)$ não é interessante que z tenha variância 1

- Porém, agora não necessariamente queremos média 0 e variância 1
- EX.: Na sigmoide $\sigma(z)$ não é interessante que z tenha variância 1

Given some intermediate values in NN: $z^{(1)}, \dots, z^{(m)}$, here, $z^{(i)} = z^{[l](i)}$ in the layer l

$$\mu = \frac{1}{m} \sum_i z^{(i)}$$

$$\sigma^2 = \frac{1}{m} \sum_i (z^{(i)} - \mu)^2$$

$$z_{norm}^{(i)} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$\tilde{z}^{(i)} = \gamma z_{norm}^{(i)} + \beta$$

Use $\tilde{z}^{[l](i)}$ instead of $z^{[l](i)}$

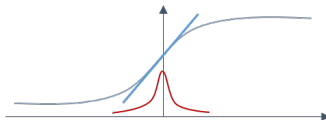
$$\text{if } \gamma = \sqrt{\sigma^2 + \epsilon}$$

$$\beta = \mu$$

then

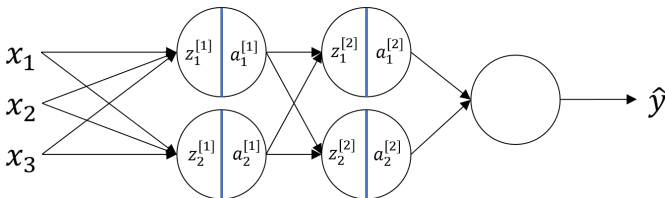
$$\tilde{z}^{(i)} = z^{(i)}$$

(γ, β is learnable parameters)



Batch Norm na Rede Neural

- Aplicado entre a soma linear z e a ativação a



$$X \xrightarrow{w^{[1],b^{[1]}}} z^{[1]} \xrightarrow[\text{BN}]{\beta^{[1],\gamma^{[1]}}} \tilde{z}^{[1]} \rightarrow a^{[1]} = g^{[1]}(\tilde{z}^{[1]}) \xrightarrow{w^{[2],b^{[2]}}} z^{[2]} \xrightarrow[\text{BN}]{\beta^{[2],\gamma^{[2]}}} \tilde{z}^{[2]} \rightarrow a^{[2]} \dots$$

Parameters:

$$w^{[1],b^{[1]},w^{[2],b^{[2]},\dots,w^{[L],b^{[L]}}$$

$$\beta^{[1],\gamma^{[1]},\beta^{[2],\gamma^{[2]},\dots,\beta^{[L],\gamma^{[L]}}$$

$$d\beta^{[l]} \longrightarrow \beta^{[l]} = \beta^{[l]} - \alpha d\beta^{[l]}$$

tf.nn.batch_normalization

Batch Norm na Rede Neural

- NOTA 1: β aqui não tem nada a ver com o β no Adam, Momento e RMSprop
- NOTA 2: não precisa implementar o passo (camada) do batch norm; *frameworks* de *deep learning* têm tudo pronto
- NOTA 3: β e γ podem ser aprendidos pelo gradiente descendente padrão ou RMSprop, Adam, etc.

Batch Norm na Rede Neural

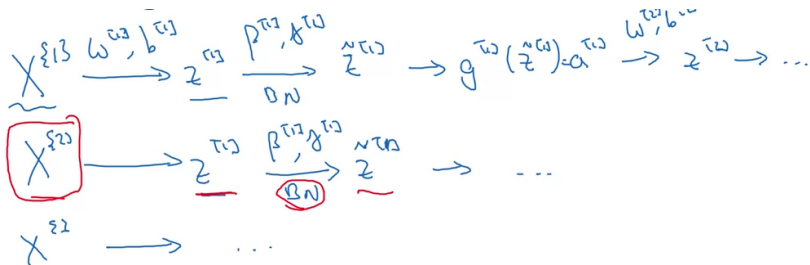
- NOTA 1: β aqui não tem nada a ver com o β no Adam, Momento e RMSprop
- NOTA 2: não precisa implementar o passo (camada) do batch norm; *frameworks* de *deep learning* têm tudo pronto
- NOTA 3: β e γ podem ser aprendidos pelo gradiente descendente padrão ou RMSprop, Adam, etc.

Batch Norm na Rede Neural

- NOTA 1: β aqui não tem nada a ver com o β no Adam, Momento e RMSprop
- NOTA 2: não precisa implementar o passo (camada) do batch norm; *frameworks* de *deep learning* têm tudo pronto
- NOTA 3: β e γ podem ser aprendidos pelo gradiente descendente padrão ou RMSprop, Adam, etc.

Batch Norm em Mini-Batches

- Na prática, todas as médias, variâncias e atualizações são feitas sobre cada mini-batch $X^{(i)}$



Batch Norm e Bias

- No batch norm não faz sentido manter o termo constante de bias b na variável z
- Como em $\tilde{z}$ subtrai-se a constante μ (que também é aprendida), eles se cancelam
- Dimensões dos parâmetros treináveis $z^{[l]}$, $\beta^{[l]}$ e $\gamma^{[l]}$: $(n^{[l]}, 1)$

Batch Norm e Bias

- No batch norm não faz sentido manter o termo constante de bias b na variável z
- Como em $\tilde{z}$ subtrai-se a constante μ (que também é aprendida), eles se cancelam

Parâmetros: $w^{[L]}$, ~~$b^{[L]}$~~ , $\beta^{[L]}$, $\gamma^{[L]}$.

$$\rightarrow \underline{z^{[L]}} = w^{[L]} a^{[L-1]} + \boxed{\cancel{b^{[L]}}}$$

↑

$$z^{[L]} = w^{[L]} a^{[L-1]}$$

$$\rightarrow \frac{z^{[L]}}{z_{\text{norm}}^{[L]}} = \gamma^{[L]} z_{\text{norm}}^{[L]} + \boxed{\beta^{[L]}} \leftarrow \text{Andri}$$

- Dimensões dos parâmetros treináveis $z^{[l]}$, $\beta^{[l]}$ e $\gamma^{[l]}$: $(n^{[l]}, 1)$

Batch Norm e Bias

- No batch norm não faz sentido manter o termo constante de bias b na variável z
- Como em $\tilde{z}$ subtrai-se a constante μ (que também é aprendida), eles se cancelam

Parâmetros: $w^{[L]}$, ~~$b^{[L]}$~~ , $\beta^{[L]}$, $\gamma^{[L]}$.

$$\rightarrow \underline{z^{[L]}} = w^{[L]} a^{[L-1]} + \boxed{\cancel{b^{[L]}}}$$

↑

$$z^{[L]} = w^{[L]} a^{[L-1]}$$

$$\rightarrow \frac{z^{[L]}}{z_{norm}^{[L]}} = \gamma^{[L]} z_{norm}^{[L]} + \boxed{\beta^{[L]}} \leftarrow \text{Andri}$$

- Dimensões dos parâmetros treináveis $z^{[l]}$, $\beta^{[l]}$ e $\gamma^{[l]}$: $(n^{[l]}, 1)$

Algoritmo Geral

for $t = 1 \dots \text{num Mini Batches}$

Compute forward pass on X^{t+1} .

In each hidden layer, use BN to replace $\underline{z}^{(t)}$ with $\hat{\underline{z}}^{(t)}$.

Use backprop to compute $\underline{dw}^{(t)}$, ~~$d\beta^{(t)}$~~ , $\underline{d\beta}^{(t)}$, $\underline{df}^{(t)}$

Update pointers $\left. \begin{aligned} w^{(t)} &:= w^{(t)} - \alpha \underline{dw}^{(t)} \\ \beta^{(t)} &:= \beta^{(t)} - \alpha \underline{d\beta}^{(t)} \\ f^{(t)} &:= \dots \end{aligned} \right\} \leftarrow$

Works w/ momentum, RMSprop, Adam.

Por que Funciona?

- Mesma intuição da normalização das entradas: todos os atributos na mesma faixa de variação acelera o aprendizado
- Ademais, ele faz com que pesos em camadas mais avançadas (e.g., camada 10) sejam mais robustos a variações nas primeiras camadas (e.g., camada 1)
- Deslocamento na distribuição das entradas: **covariate shift**

Por que Funciona?

- Mesma intuição da normalização das entradas: todos os atributos na mesma faixa de variação acelera o aprendizado
- Ademais, ele faz com que pesos em camadas mais avançadas (e.g., camada 10) sejam mais robustos a variações nas primeiras camadas (e.g., camada 1)
- Deslocamento na distribuição das entradas: **covariate shift**

Por que Funciona?

- Mesma intuição da normalização das entradas: todos os atributos na mesma faixa de variação acelera o aprendizado
- Ademais, ele faz com que pesos em camadas mais avançadas (e.g., camada 10) sejam mais robustos a variações nas primeiras camadas (e.g., camada 1)
- Deslocamento na distribuição das entradas: **covariate shift**

Por que Funciona?

- Suponha uma rede rasa ou regressão logística treinada para identificar gatos
- Porém usando apenas imagens de gatos pretos no treino
- No teste com gatos coloridos, o modelo não vai funcionar bem
- Mesmo que a fronteira de decisão nos dois cenários seja a mesma, o modelo precisa ser retreinado

Por que Funciona?

- Suponha uma rede rasa ou regressão logística treinada para identificar gatos
- Porém usando apenas imagens de gatos pretos no treino
- No teste com gatos coloridos, o modelo não vai funcionar bem
- Mesmo que a fronteira de decisão nos dois cenários seja a mesma, o modelo precisa ser retreinado

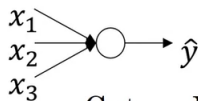
Por que Funciona?

- Suponha uma rede rasa ou regressão logística treinada para identificar gatos
- Porém usando apenas imagens de gatos pretos no treino
- No teste com gatos coloridos, o modelo não vai funcionar bem
- Mesmo que a fronteira de decisão nos dois cenários seja a mesma, o modelo precisa ser retreinado

Por que Funciona?

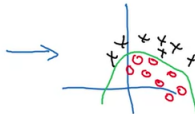
- Suponha uma rede rasa ou regressão logística treinada para identificar gatos
- Porém usando apenas imagens de gatos pretos no treino
- No teste com gatos coloridos, o modelo não vai funcionar bem
- Mesmo que a fronteira de decisão nos dois cenários seja a mesma, o modelo precisa ser retreinado

Por que Funciona?



Cat

Non-Cat

 $y = 1$ ✓ $y = 0$  $y = 1$ ✓ $y = 0$ 

"Covariate shift"

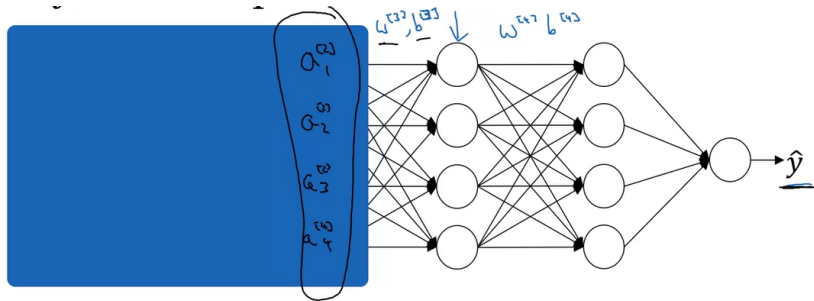
 $\underline{x} \rightarrow y$

Covariate Shift na Rede Profunda

- Se focarmos, por exemplo, na camada 3, podemos apagar as camadas anteriores
- Como se recebesse *features* de entrada e ajustasse pesos para chegar na saída $\hat{y}$
- Porém, essas entradas não são mais fixas, mas sim as ativações $a^{[2]}$ alteradas em cada época

Covariate Shift na Rede Profunda

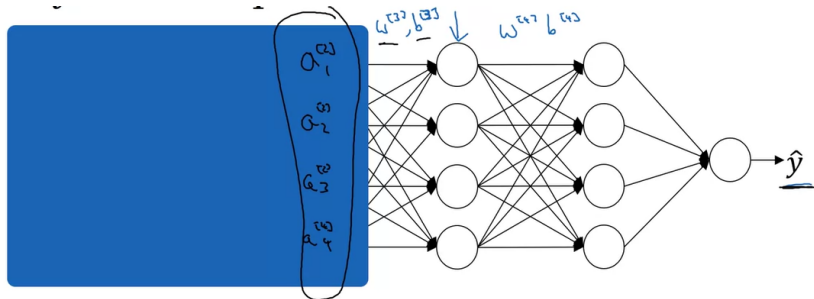
- Se focarmos, por exemplo, na camada 3, podemos apagar as camadas anteriores
- Como se recebesse *features* de entrada e ajustasse pesos para chegar na saída $\hat{y}$



- Porém, essas entradas não são mais fixas, mas sim as ativações $a^{[2]}$ alteradas em cada época

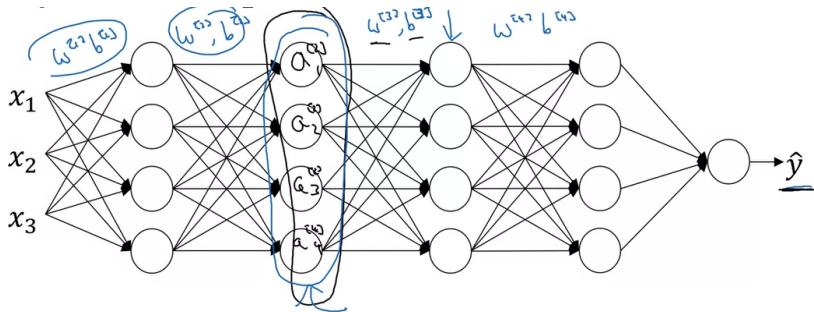
Covariate Shift na Rede Profunda

- Se focarmos, por exemplo, na camada 3, podemos apagar as camadas anteriores
- Como se recebesse *features* de entrada e ajustasse pesos para chegar na saída $\hat{y}$



- Porém, essas entradas não são mais fixas, mas sim as ativações $a^{[2]}$ alteradas em cada época

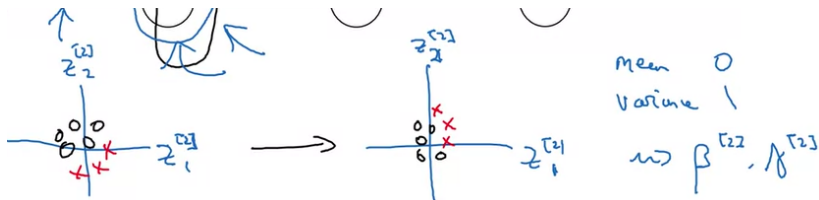
Covariate Shift na Rede Profunda



- *Covariate shift* se propagando pelas camadas: toda variação nas camadas iniciais interferindo nas próximas

Papel do Batch Norm

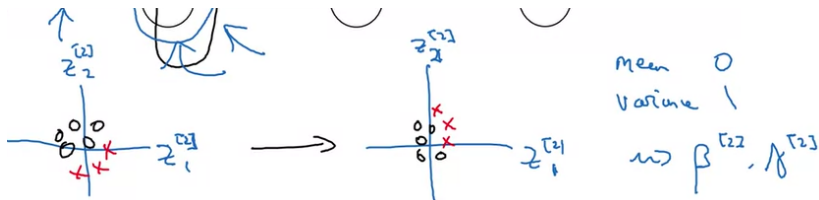
- Batch norm controla esse deslocamento da distribuição forçando a média e variância fixados (controlado por β e γ)



- Aprendizado mais independente entre as camadas

Papel do Batch Norm

- Batch norm controla esse deslocamento da distribuição forçando a média e variância fixados (controlado por β e γ)



- Aprendizado mais independente entre as camadas

Batch Norm e Regularização

- Leve regularização: em cada *mini-batch* usa média e variância calculadas no próprio *mini-batch*
- Assim, essa média e variância têm um nível maior de ruído, também adicionado a $\hat{z}^{[l]}$

Adição de ruído em cada ativação, similar ao *dropout*

- NOTA: Quanto maior o tamanho do mini-batch (e.g., 512 em vez de 64), menor é o efeito regularizador
- IMPORTANTE: Batch norm **não** é um método de regularização, isso é só um efeito colateral!

Batch Norm e Regularização

- Leve regularização: em cada *mini-batch* usa média e variância calculadas no próprio *mini-batch*
- Assim, essa média e variância têm um nível maior de ruído, também adicionado a $\tilde{z}^{[l]}$

Adição de ruído em cada ativação, similar ao *dropout*

- NOTA: Quanto maior o tamanho do mini-batch (e.g., 512 em vez de 64), menor é o efeito regularizador
- IMPORTANTE: Batch norm **não** é um método de regularização, isso é só um efeito colateral!

Batch Norm e Regularização

- Leve regularização: em cada *mini-batch* usa média e variância calculadas no próprio *mini-batch*
- Assim, essa média e variância têm um nível maior de ruído, também adicionado a $\tilde{z}^{[l]}$

Adição de ruído em cada ativação, similar ao *dropout*

- NOTA: Quanto maior o tamanho do mini-batch (e.g., 512 em vez de 64), menor é o efeito regularizador
- IMPORTANTE: Batch norm **não** é um método de regularização, isso é só um efeito colateral!

Batch Norm e Regularização

- Leve regularização: em cada *mini-batch* usa média e variância calculadas no próprio *mini-batch*
- Assim, essa média e variância têm um nível maior de ruído, também adicionado a $\tilde{z}^{[l]}$

Adição de ruído em cada ativação, similar ao *dropout*

- NOTA: Quanto maior o tamanho do mini-batch (e.g., 512 em vez de 64), menor é o efeito regularizador
- IMPORTANTE: Batch norm **não** é um método de regularização, isso é só um efeito colateral!

Batch Norm no Teste

- No treino, um *mini-batch* de cada vez; e no teste?
- Treino:

Batch Norm no Teste

- No treino, um *mini-batch* de cada vez; e no teste?
- Treino:

$$\begin{aligned} \rightarrow \mu &= \frac{1}{m} \sum_i z^{(i)} \\ \rightarrow \sigma^2 &= \frac{1}{m} \sum_i (z^{(i)} - \mu)^2 \\ \rightarrow z_{\text{norm}}^{(i)} &= \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}} \\ \rightarrow \tilde{z}^{(i)} &= \gamma z_{\text{norm}}^{(i)} + \beta \end{aligned}$$

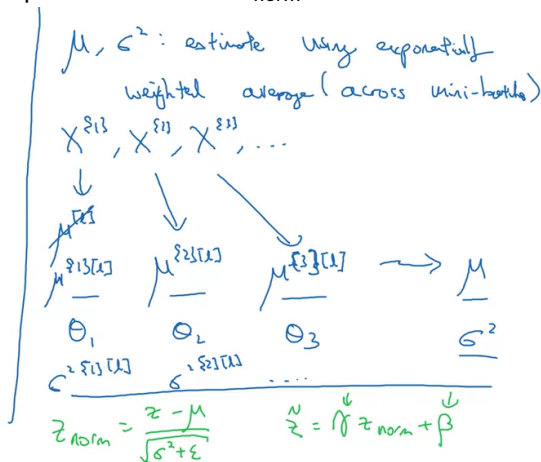
em que m no caso é o tamanho do *mini-batch*

Batch Norm no Teste

- No teste, calcula-se uma média exponencialmente ponderada (*running average*) sobre μ e σ em cada *mini-batch*
- Usa esse valor para o cálculo de z_{norm}

Batch Norm no Teste

- No teste, calcula-se uma média exponencialmente ponderada (*running average*) sobre μ e σ em cada *mini-batch*
- Usa esse valor para o cálculo de z_{norm}



Outline

- 1 Ajuste de Hiperparâmetros
- 2 Batch Normalization
- 3 Classificação Multi-classes**
- 4 Frameworks de Deep Learning

Regressão Softmax

- Generalização da regressão logística para múltiplas classes
- Reconhecer gatos (1), cachorros (2), pintinhos (3) e outros (0)
- C é o número de classes
- Saída da rede neural vai ter C unidades, cada uma correspondendo à probabilidade daquela classe dado x (soma 1)

Regressão Softmax

- Generalização da regressão logística para múltiplas classes
- Reconhecer gatos (1), cachorros (2), pintinhos (3) e outros (0)



3



1



2



0



3



2



0



1

- C é o número de classes
- Saída da rede neural vai ter C unidades, cada uma correspondendo à probabilidade daquela classe dado x (soma 1)

Regressão Softmax

- Generalização da regressão logística para múltiplas classes
- Reconhecer gatos (1), cachorros (2), pintinhos (3) e outros (0)



3



1



2



0



3



2



0



1

- C é o número de classes
- Saída da rede neural vai ter C unidades, cada uma correspondendo à probabilidade daquela classe dado x (soma 1)

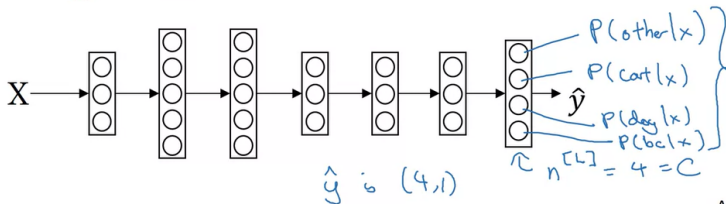
Regressão Softmax

- Generalização da regressão logística para múltiplas classes
- Reconhecer gatos (1), cachorros (2), pintinhos (3) e outros (0)

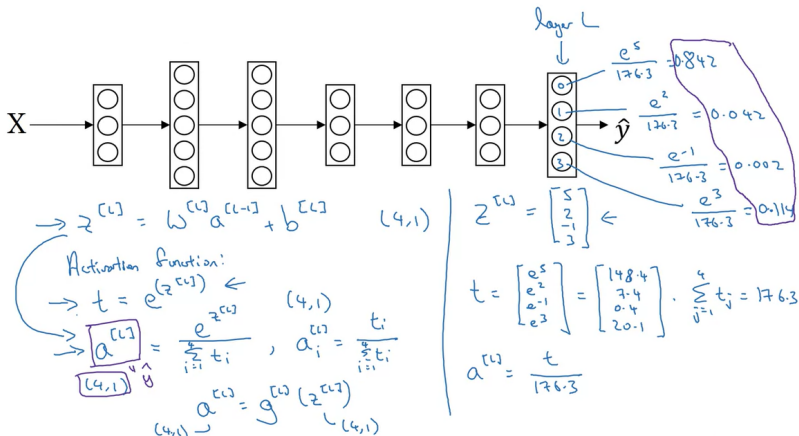


- C é o número de classes
- Saída da rede neural vai ter C unidades, cada uma correspondendo à probabilidade daquela classe dado x (soma 1)

$$C = \#classes = 4 \quad (0, \dots, 3)$$

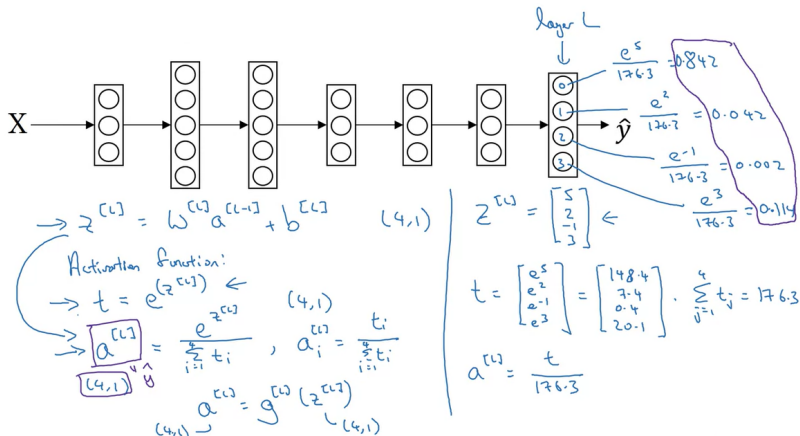


Camada Softmax



- NOTA: Diferente das funções de ativação vistas, a softmax recebe e retorna um vetor e não um número real

Camada Softmax



- NOTA: Diferente das funções de ativação vistas, a softmax recebe e retorna um vetor e não um número real

Código

```
def softmax(z):
    return np.exp(z) / sum(np.exp(z))
```

```
z = [1, 0.5, -2, 1, 3]
```

```
print(softmax(z))
```

```
# array([0.09954831, 0.0603791 , 0.00495622, 0.09954831,
        0.73556806])
```

- CURIOSIDADE: *softmax* é uma alternativa ao *hardmax*, que gera um vetor em que o maior elemento é mapeado para 1 e todos os demais para zero

$$\begin{aligned}
 & \text{Handwritten: } (4,1) \\
 & z^{[L]} = \begin{bmatrix} 5 \\ 2 \\ -1 \\ 3 \end{bmatrix} \quad t = \begin{bmatrix} e^5 \\ e^2 \\ e^{-1} \\ e^3 \end{bmatrix} \\
 & \text{Handwritten: "Soft max"} \\
 & a^{[L]} = g^{[L]}(z^{[L]}) = \begin{bmatrix} e^5 / (e^5 + e^2 + e^{-1} + e^3) \\ e^2 / (e^5 + e^2 + e^{-1} + e^3) \\ e^{-1} / (e^5 + e^2 + e^{-1} + e^3) \\ e^3 / (e^5 + e^2 + e^{-1} + e^3) \end{bmatrix} = \begin{bmatrix} 0.842 \\ 0.042 \\ 0.002 \\ 0.114 \end{bmatrix}
 \end{aligned}$$

Handwritten: $C=4$ $g^{[L]}(\cdot)$

Handwritten: "hard max"

$$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Código

```
def softmax(z):
    return np.exp(z) / sum(np.exp(z))
```

```
z = [1, 0.5, -2, 1, 3]
```

```
print(softmax(z))
```

```
# array([0.09954831, 0.0603791 , 0.00495622, 0.09954831,
        0.73556806])
```

- CURIOSIDADE: *softmax* é uma alternativa ao *hardmax*, que gera um vetor em que o maior elemento é mapeado para 1 e todos os demais para zero

$(4,1)$

$$z^{[L]} = \begin{bmatrix} 5 \\ 2 \\ -1 \\ 3 \end{bmatrix} \quad t = \begin{bmatrix} e^5 \\ e^2 \\ e^{-1} \\ e^3 \end{bmatrix}$$

"Soft max"

$$a^{[L]} = g^{[L]}(z^{[L]}) = \begin{bmatrix} e^5 / (e^5 + e^2 + e^{-1} + e^3) \\ e^2 / (e^5 + e^2 + e^{-1} + e^3) \\ e^{-1} / (e^5 + e^2 + e^{-1} + e^3) \\ e^3 / (e^5 + e^2 + e^{-1} + e^3) \end{bmatrix} = \begin{bmatrix} 0.842 \\ 0.042 \\ 0.002 \\ 0.114 \end{bmatrix}$$

$C=4$

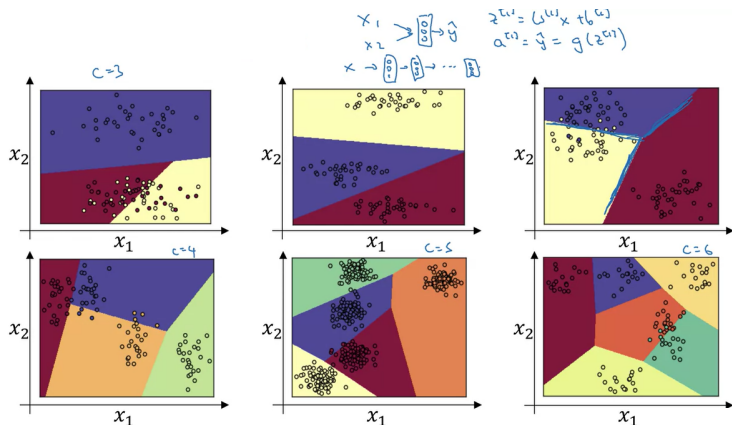
$g^{[L]}(\cdot)$

"hard max"

$$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Fronteira de Decisão

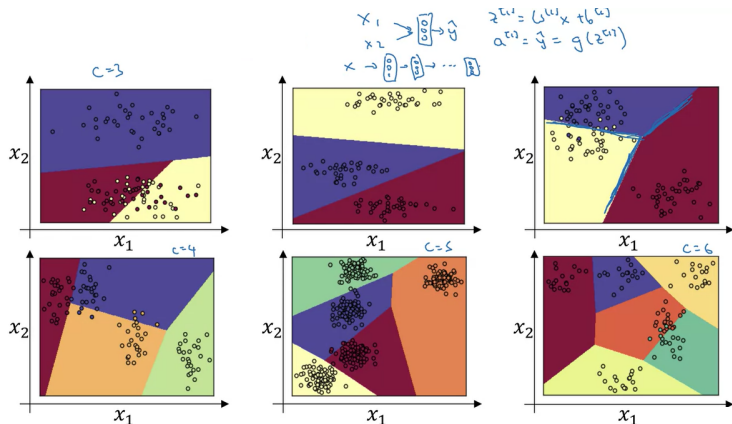
Exemplo sem camada escondida:



NOTA: Softmax com 2 camadas é exatamente equivalente à regressão logística

Fronteira de Decisão

Exemplo sem camada escondida:



NOTA: Softmax com 2 camadas é exatamente equivalente à regressão logística

Treinamento

- Classe real é representada por um vetor em que apenas a posição da classe é 1 e todas as demais são 0

$$\begin{array}{l}
 (4,1) \\
 y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \\
 \text{cat} \\
 y_2 = 1 \\
 y_1 = y_3 = y_4 = 0
 \end{array}$$

$$\begin{array}{l}
 (4,1) \\
 a^{(1)} = y = \begin{bmatrix} 0.3 \\ 0.2 \\ 0.1 \\ 0.4 \end{bmatrix}
 \end{array}$$

$$C = 4$$

- Loss function:*

$$\mathcal{L}(\hat{y}, y) = - \sum_{j=1}^4 y_j \log \hat{y}_j$$

No exemplo acima:

$$\mathcal{L}(\hat{y}, y) = - \log \hat{y}_2$$

- Minimizar $\mathcal{L}(\hat{y}, y)$ implica maximizar $\hat{y}_2$, que é exatamente a probabilidade de a imagem ser um gato

Treinamento

- Classe real é representada por um vetor em que apenas a posição da classe é 1 e todas as demais são 0

$$\begin{matrix} (4,1) \\ y = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \end{matrix} \quad \begin{matrix} \text{--- cat} \\ y_2 = 1 \\ y_1 = y_3 = y_4 = 0 \end{matrix}$$

$$\begin{matrix} (4,1) \\ a^{(1)} = y = \begin{bmatrix} 0.3 \\ 0.2 \\ 0.1 \\ 0.4 \end{bmatrix} \end{matrix} \quad \leftarrow$$

$$C = 4$$

- Loss function:*

$$\mathcal{L}(\hat{y}, y) = - \sum_{j=1}^4 y_j \log \hat{y}_j$$

No exemplo acima:

$$\mathcal{L}(\hat{y}, y) = - \log \hat{y}_2$$

- Minimizar $\mathcal{L}(\hat{y}, y)$ implica maximizar $\hat{y}_2$, que é exatamente a probabilidade de a imagem ser um gato

Treinamento

- Classe real é representada por um vetor em que apenas a posição da classe é 1 e todas as demais são 0

$$\begin{array}{l}
 (4,1) \\
 y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \leftarrow \text{cat} \\
 y_2 = 1 \\
 y_1 = y_3 = y_4 = 0
 \end{array}$$

$$\begin{array}{l}
 (4,1) \\
 a^{(1)} = y = \begin{bmatrix} 0.3 \\ 0.2 \\ 0.1 \\ 0.4 \end{bmatrix}
 \end{array}$$

$$C = 4$$

- Loss function:

$$\mathcal{L}(\hat{y}, y) = - \sum_{j=1}^4 y_j \log \hat{y}_j$$

No exemplo acima:

$$\mathcal{L}(\hat{y}, y) = - \log \hat{y}_2$$

- Minimizar $\mathcal{L}(\hat{y}, y)$ implica maximizar $\hat{y}_2$, que é exatamente a probabilidade de a imagem ser um gato

- Função de custo:

$$J(W^{[1]}, b^{[1]}, \dots) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

- Vetorização:

- No *backpropagation*, a única mudança é a derivada $dz^{[l]}$:

$$dz^{[l]} = \frac{\partial J}{\partial z^{[l]}} = \hat{y} - y$$

- *Frameworks* cuidarão do *backward* automaticamente

- Função de custo:

$$J(W^{[1]}, b^{[1]}, \dots) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

- Vetorização:

$$\begin{aligned} Y &= [y^{(1)} \ y^{(2)} \ \dots \ y^{(m)}] & \hat{Y} &= [\hat{y}^{(1)} \ \dots \ \hat{y}^{(m)}] \\ &= \begin{bmatrix} 0 & 0 & 1 & \dots \\ 1 & 0 & 0 & \dots \\ 0 & 1 & 0 & \dots \\ 0 & 0 & 0 & \dots \end{bmatrix} & &= \begin{bmatrix} 0.3 & \dots \\ 0.2 & \dots \\ 0.1 & \dots \\ 0.4 & \dots \end{bmatrix} \\ (4, m) & & (4, m) \end{aligned}$$

- No *backpropagation*, a única mudança é a derivada $dz^{[l]}$:

$$dz^{[l]} = \frac{\partial J}{\partial z^{[l]}} = \hat{y} - y$$

- *Frameworks* cuidarão do *backward* automaticamente

- Função de custo:

$$J(W^{[1]}, b^{[1]}, \dots) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

- Vetorização:

$$\begin{aligned} Y &= [y^{(1)} \ y^{(2)} \ \dots \ y^{(m)}] & \hat{Y} &= [\hat{y}^{(1)} \ \dots \ \hat{y}^{(m)}] \\ &= \begin{bmatrix} 0 & 0 & 1 & \dots \\ 1 & 0 & 0 & \dots \\ 0 & 1 & 0 & \dots \\ 0 & 0 & 0 & \dots \end{bmatrix} & &= \begin{bmatrix} 0.3 & \dots \\ 0.2 & \dots \\ 0.1 & \dots \\ 0.4 & \dots \end{bmatrix} \\ (4, m) & & (4, m) \end{aligned}$$

- No *backpropagation*, a única mudança é a derivada $dz^{[l]}$:

$$dz^{[l]} = \frac{\partial J}{\partial z^{[l]}} = \hat{y} - y$$

- *Frameworks* cuidarão do *backward* automaticamente

- Função de custo:

$$J(W^{[1]}, b^{[1]}, \dots) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

- Vetorização:

$$\begin{array}{ll}
 \underline{Y} = [y^{(1)} \ y^{(2)} \ \dots \ y^{(m)}] & \underline{\hat{Y}} = [\hat{y}^{(1)} \ \dots \ \hat{y}^{(m)}] \\
 = \begin{bmatrix} 0 & 0 & 1 & \dots \\ 1 & 0 & 0 & \dots \\ 0 & 1 & 0 & \dots \\ 0 & 0 & 0 & \dots \end{bmatrix} & = \begin{bmatrix} 0.3 & \dots \\ 0.2 & \dots \\ 0.1 & \dots \\ 0.4 & \dots \end{bmatrix} \\
 (4, m) & (4, m)
 \end{array}$$

- No *backpropagation*, a única mudança é a derivada $dz^{[l]}$:

$$dz^{[l]} = \frac{\partial J}{\partial dz^{[l]}} = \hat{y} - y$$

- *Frameworks* cuidarão do *backward* automaticamente

Outline

- 1 Ajuste de Hiperparâmetros
- 2 Batch Normalization
- 3 Classificação Multi-classes
- 4 Frameworks de Deep Learning**

- Implementar modelos grandes como redes convolucionais ou recorrentes do zero não é nada prático
- Ótimos *frameworks* para fazer isso
- Mesmo sabendo como multiplicar matrizes, não vai fazer isso na mão
- EX.: Caffe, CNTK, DL4j, Lasagne, Mxnet, PaddlePaddle, **PyTorch**, **Keras**, **TensorFlow**

- Implementar modelos grandes como redes convolucionais ou recorrentes do zero não é nada prático
- Ótimos *frameworks* para fazer isso
- Mesmo sabendo como multiplicar matrizes, não vai fazer isso na mão
- EX.: Caffe, CNTK, DL4j, Lasagne, Mxnet, PaddlePaddle, **PyTorch**, **Keras**, **TensorFlow**

- Implementar modelos grandes como redes convolucionais ou recorrentes do zero não é nada prático
- Ótimos *frameworks* para fazer isso
- Mesmo sabendo como multiplicar matrizes, não vai fazer isso na mão
- EX.: Caffe, CNTK, DL4j, Lasagne, Mxnet, PaddlePaddle, **PyTorch**, **Keras**, **TensorFlow**

- Implementar modelos grandes como redes convolucionais ou recorrentes do zero não é nada prático
- Ótimos *frameworks* para fazer isso
- Mesmo sabendo como multiplicar matrizes, não vai fazer isso na mão
- EX.: Caffe, CNTK, DL4j, Lasagne, Mxnet, PaddlePaddle, **PyTorch**, **Keras**, **TensorFlow**

Critérios de Escolha

- Facilidade de programação (desenvolvimento e *deployment*)
- Velocidade de execução (modelos grandes, com muitos dados)
- Ser verdadeiramente aberto (código aberto + boa governança): histórico de permanecer *open*.

Critérios de Escolha

- Facilidade de programação (desenvolvimento e *deployment*)
- Velocidade de execução (modelos grandes, com muitos dados)
- Ser verdadeiramente aberto (código aberto + boa governança): histórico de permanecer *open*.

Critérios de Escolha

- Facilidade de programação (desenvolvimento e *deployment*)
- Velocidade de execução (modelos grandes, com muitos dados)
- Ser verdadeiramente aberto (código aberto + boa governança): histórico de permanecer *open*.

TensorFlow

Minimizar uma função usando Adam ($\alpha = 0.01$):

```
import numpy as np
import tensorflow as tf

coefficients = np.array([[1], [-20], [25]])
w = tf.Variable([0], dtype=tf.float32)
x = tf.placeholder(tf.float32, [3, 1])
cost = x[0][0]*w**2 + x[1][0]*w + x[2][0]      # (w-5)**2
train = tf.train.GradientDescentOptimizer(0.01).minimize(cost)

init = tf.global_variables_initializer()
session = tf.Session()
session.run(init)
print(session.run(w))

for i in range(1000):
    session.run(train, feed_dict={x: coefficients})
    print(session.run(w))
```

- Duas classes principais de objetos: **Tensores** e **Operadores**

- 1 Criar o grafo contendo Tensores (*Variables, Placeholders, ...*) e Operações (*tf.matmul, tf.add, ...*)
 - 2 Crie a sessão
 - 3 Inicialize a sessão
 - 4 Rode a sessão para executar o grafo
- Definimos explicitamente apenas o *forward*; gradientes calculados automaticamente
 - Parâmetros treináveis são *Variable*, não treináveis usam *placeholder*
 - O otimizador é mudado com uma linha

- Duas classes principais de objetos: **Tensores** e **Operadores**
 - ① Criar o grafo contendo Tensores (*Variables*, *Placeholders*, ...) e Operações (`tf.matmul`, `tf.add`, ...)
 - ② Crie a sessão
 - ③ Inicialize a sessão
 - ④ Rode a sessão para executar o grafo
- Definimos explicitamente apenas o *forward*; gradientes calculados automaticamente
- Parâmetros treináveis são *Variable*, não treináveis usam *placeholder*
- O otimizador é mudado com uma linha

- Duas classes principais de objetos: **Tensores** e **Operadores**
 - ① Criar o grafo contendo Tensores (*Variables*, *Placeholders*, ...) e Operações (*tf.matmul*, *tf.add*, ...)
 - ② Crie a sessão
 - ③ Inicialize a sessão
 - ④ Rode a sessão para executar o grafo
- Definimos explicitamente apenas o *forward*; gradientes calculados automaticamente
- Parâmetros treináveis são *Variable*, não treináveis usam *placeholder*
- O otimizador é mudado com uma linha

- Duas classes principais de objetos: **Tensores** e **Operadores**
 - 1 Criar o grafo contendo Tensores (*Variables*, *Placeholders*, ...) e Operações (*tf.matmul*, *tf.add*, ...)
 - 2 Crie a sessão
 - 3 Inicialize a sessão
 - 4 Rode a sessão para executar o grafo
- Definimos explicitamente apenas o *forward*; gradientes calculados automaticamente
- Parâmetros treináveis são *Variable*, não treináveis usam *placeholder*
- O otimizador é mudado com uma linha

- Duas classes principais de objetos: **Tensores** e **Operadores**
 - 1 Criar o grafo contendo Tensores (*Variables*, *Placeholders*, ...) e Operações (*tf.matmul*, *tf.add*, ...)
 - 2 Crie a sessão
 - 3 Inicialize a sessão
 - 4 Rode a sessão para executar o grafo
- Definimos explicitamente apenas o *forward*; gradientes calculados automaticamente
- Parâmetros treináveis são *Variable*, não treináveis usam *placeholder*
- O otimizador é mudado com uma linha

- Duas classes principais de objetos: **Tensores** e **Operadores**
 - 1 Criar o grafo contendo Tensores (*Variables*, *Placeholders*, ...) e Operações (*tf.matmul*, *tf.add*, ...)
 - 2 Crie a sessão
 - 3 Inicialize a sessão
 - 4 Rode a sessão para executar o grafo
- Definimos explicitamente apenas o *forward*; gradientes calculados automaticamente
- Parâmetros treináveis são *Variable*, não treináveis usam *placeholder*
- O otimizador é mudado com uma linha

- Duas classes principais de objetos: **Tensores** e **Operadores**
 - ① Criar o grafo contendo Tensores (*Variables*, *Placeholders*, ...) e Operações (*tf.matmul*, *tf.add*, ...)
 - ② Crie a sessão
 - ③ Inicialize a sessão
 - ④ Rode a sessão para executar o grafo
- Definimos explicitamente apenas o *forward*; gradientes calculados automaticamente
- Parâmetros treináveis são *Variable*, não treináveis usam *placeholder*
- O otimizador é mudado com uma linha

- Duas classes principais de objetos: **Tensores** e **Operadores**
 - 1 Criar o grafo contendo Tensores (*Variables*, *Placeholders*, ...) e Operações (*tf.matmul*, *tf.add*, ...)
 - 2 Crie a sessão
 - 3 Inicialize a sessão
 - 4 Rode a sessão para executar o grafo
- Definimos explicitamente apenas o *forward*; gradientes calculados automaticamente
- Parâmetros treináveis são *Variable*, não treináveis usam *placeholder*
- O otimizador é mudado com uma linha